

Python Practice Pack: Loops, Conditionals & String Manipulation

Concept Guide + Practice Questions

- [Part 1: Core Concepts — Syntax & Logic](#)
- [Part 2: Important String Functions \(Syntax + Examples\)](#)
- [Part 3: Practice Questions \(Medium → Hard\)](#)

Part 1: Core Concepts — Syntax & Logic

1.1 Conditional Statements

if / elif / else

```
if condition1:
    # runs if condition1 is True
elif condition2:
    # runs if condition1 is False AND condition2 is True
else:
    # runs if none of the above conditions are True
```

Logic: Python checks conditions **top to bottom** and executes the **first** block whose condition is `True`, then skips the rest. If none match, `else` runs. Only **one** block ever executes.

Nested conditionals:

```
if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Too young")
```

Ternary (one-line) conditional:

```
status = "Adult" if age >= 18 else "Minor"
```

1.2 Switch-Case Equivalent in Python

Python (3.10+) has `match-case`, and older code often simulates switch-case with dictionaries.

match-case

```
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case _:
        pass
```

```
print("Invalid day") # '_' is the default/wildcard case
```

Logic: Works like `if-elif-else` but is designed for matching a value against multiple fixed options. `case _:` is the catch-all (like `else`).

Dictionary-based dispatch (classic Python “switch”)

```
def monday():
    return "Start of week"
def tuesday():
    return "Second day"

day_map = {
    1: monday,
    2: tuesday
}

result = day_map.get(day, lambda: "Invalid day")()
```

Logic: Each “case” is a dictionary key mapped to a function/value. `.get(key, default)` avoids `KeyError` if the key doesn’t exist — acting like the `default` case.

1.3 Loops

for loop

```
for item in sequence:
    # runs once per item in sequence (string, list, range, etc.)
```

```
for i in range(start, stop, step):
    # start is inclusive, stop is exclusive
```

while loop

```
while condition:
    # runs repeatedly AS LONG AS condition is True
```

Logic difference: Use `for` when you know *what* you’re iterating over (a sequence, or a fixed count). Use `while` when looping depends on a *condition* that changes unpredictably (e.g., user input, search until found).

break, continue, and loop else

```
for item in sequence:
    if item == target:
        break # exits the loop immediately
    if item is invalid:
        continue # skips to the next iteration, rest of code below is not run
else:
    print("Loop finished without break") # runs only if loop completed WITHOUT break
```

Logic: `break` stops the loop entirely. `continue` skips only the current iteration. The `else` on a loop is a lesser-known feature — it runs only when the loop finishes naturally (no `break` occurred).

1.4 String Manipulation Basics

Indexing and slicing

```
s = "python"
s[0]      # 'p'    (first character, index starts at 0)
s[-1]     # 'n'    (last character, negative indexing)
s[1:4]    # 'yth'  (from index 1 up to, NOT including, index 4)
s[::-1]   # 'nohtyp' (reversed string using step = -1)
s[::2]    # 'pto'  (every 2nd character)
```

Logic: Slicing syntax is `s[start:stop:step]`. `start` is inclusive, `stop` is exclusive. Omitting a value uses the default (start of string, end of string, or step 1).

Iterating over strings

```
for char in "hello":
    print(char)    # prints one character per iteration
```

Concatenation and repetition

```
"Hello" + " " + "World"    # "Hello World"
"ab" * 3                   # "ababab"
```

Immutability

Strings **cannot** be modified in place — every operation (`.upper()`, slicing, concatenation) returns a **new** string. `s[0] = 'X'` raises a `TypeError`.

Part 2: Important String Functions (Syntax + Examples)

Function	Syntax	What it does	Example
<code>len()</code>	<code>len(s)</code>	Returns length of string	<code>len("hello") → 5</code>
<code>upper()</code>	<code>s.upper()</code>	Converts to uppercase	<code>"hi".upper() → "HI"</code>
<code>lower()</code>	<code>s.lower()</code>	Converts to lowercase	<code>"HI".lower() → "hi"</code>
<code>strip()</code>	<code>s.strip(chars=None)</code>	Removes leading/trailing whitespace (or given chars)	<code>" hi ".strip() → "hi"</code>
<code>lstrip()</code> / <code>rstrip()</code>	<code>s.lstrip()</code> / <code>s.rstrip()</code>	Strips only left / only right side	<code>"xxhixx".strip("x") → "hi"</code>
<code>split()</code>	<code>s.split(sep=None)</code>	Splits string into a list by delimiter (default: whitespace)	<code>"a,b,c".split(",") → ['a', 'b', 'c']</code>
<code>join()</code>	<code>sep.join(list)</code>	Joins a list of strings using <code>sep</code> as glue	<code>"-".join(['a', 'b']) → "a-b"</code>
<code>replace()</code>	<code>s.replace(old, new)</code>	Replaces all occurrences of <code>old</code> with <code>new</code>	<code>"cat".replace("c", "b") → "bat"</code>
<code>find()</code>	<code>s.find(sub)</code>	Returns index of first occurrence, or <code>-1</code> if not found	<code>"hello".find("l") → 2</code>
<code>index()</code>	<code>s.index(sub)</code>	Same as <code>find()</code> but raises <code>ValueError</code> if not found	<code>"hello".index("z") → error</code>
<code>count()</code>	<code>s.count(sub)</code>	Counts occurrences of a substring	<code>"banana".count("a") → 3</code>
<code>startswith()</code>	<code>s.startswith(prefix)</code>	Checks if string starts with	<code>"hello".startswith("he")</code>

		given text	→ True
<code>endswith()</code>	<code>s.endswith(suffix)</code>	Checks if string ends with given text	<code>"hello".endswith("lo")</code> → True
<code>isalpha()</code>	<code>s.isalpha()</code>	True if all characters are letters	<code>"abc".isalpha()</code> → True
<code>isdigit()</code>	<code>s.isdigit()</code>	True if all characters are digits	<code>"123".isdigit()</code> → True
<code>isalnum()</code>	<code>s.isalnum()</code>	True if all characters are letters or digits	<code>"a1".isalnum()</code> → True
<code>isspace()</code>	<code>s.isspace()</code>	True if string is only whitespace	<code>" ".isspace()</code> → True
<code>title()</code>	<code>s.title()</code>	Capitalizes first letter of every word	<code>"hi there".title()</code> → "Hi There"
<code>capitalize()</code>	<code>s.capitalize()</code>	Capitalizes only the first letter of the string	<code>"hi there".capitalize()</code> → "Hi there"
<code>swapcase()</code>	<code>s.swapcase()</code>	Swaps upper ↔ lower case	<code>"Hi".swapcase()</code> → "hI"
<code>zfill()</code>	<code>s.zfill(width)</code>	Pads string with leading zeros	<code>"5".zfill(3)</code> → "005"
<code>center()</code>	<code>s.center(width, fillchar)</code>	Centers string, padding with fillchar	<code>"hi".center(6, "*")</code> → "***hi**"
<code>ljust()</code> / <code>rjust()</code>	<code>s.ljust(width) / s.rjust(width)</code>	Left/right-aligns string within given width	<code>"hi".rjust(5, "0")</code> → "000hi"
<code>partition()</code>	<code>s.partition(sep)</code>	Splits into a 3-tuple: (before, sep, after) — first match only	<code>"a=b=c".partition("=")</code> → ('a', '=', 'b=c')
<code>splitlines()</code>	<code>s.splitlines()</code>	Splits string at line breaks into a list	<code>"a\nb".splitlines()</code> → ['a', 'b']
f-strings	<code>f"{var}"</code>	Embeds expressions/variables directly into a string	<code>f"{2+2}"</code> → "4"

Part 3: Practice Questions (Medium → Hard)

Each question lists its **core concept**, **difficulty**, and **hints**. Hints are lighter for medium problems and more structured (broken into steps) for hard ones — try solving before reading them.

Q1. Grade Calculator with Weighted Average (Medium — Conditionals)

Given marks in 3 subjects with weights 40%, 30%, 30%, compute the weighted average and print a grade: A (≥85), B (≥70), C (≥50), F (below 50).

Hint: Compute the weighted sum first using multiplication, then run it through an `if-elif-else` chain ordered from highest grade to lowest.

Q2. Leap Year Validator (Medium — Conditionals)

Check if a year is a leap year. Rule: divisible by 4, EXCEPT century years (divisible by 100) unless also divisible by 400.

Hint: This needs nested or combined conditions — think about what makes a year divisible by 4 “not a leap year” (the century exception), and what makes it “a leap year again” (the 400 exception).

Q3. Right-Angled Number Pyramid (Medium — Loops)

Print a pyramid pattern using nested loops for `n = 5`:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Hint: Use an outer loop for each row (1 to n) and an inner loop that runs from 1 up to the current row number.

Q4. Prime Number Checker (*Medium — Loops*)

Write a function that checks if a number is prime, using a loop (no built-in libraries).

Hint: A number is prime if no number from 2 up to its square root divides it evenly. You don't need to check all the way up to `n-1` — that's a common inefficiency worth avoiding.

Q5. Digital Root (Repeated Digit Sum) (*Medium — Loops*)

Given a number, repeatedly sum its digits until only a single digit remains (e.g., `9875 → 9+8+7+5=29 → 2+9=11 → 1+1=2`).

Hint: You'll need a loop *inside* a loop — an outer `while` that keeps repeating “as long as the number has more than one digit,” and an inner loop or logic to sum the digits of the current number.

Q6. Reverse a String Without Slicing (*Medium — String Manipulation*)

Reverse a string manually, without using `[::-1]` or `reversed()`.

Hint: Build a new empty string, and either loop through the original string backwards using indices, or loop forwards and **prepend** each character to the result instead of appending.

Q7. Vowel, Consonant, Space & Digit Counter (*Medium — String Manipulation*)

Given a string, count vowels, consonants, spaces, and digits separately in a single pass.

Hint: Loop once through the string, and inside the loop use `if-elif-else` branches: check `isspace()`, `isdigit()`, then whether the lowercased character is a vowel, else treat it as a consonant.

Q8. Pangram Checker (*Medium/Hard — String Manipulation*)

Check if a sentence is a pangram (contains every letter of the alphabet at least once), ignoring case.

Hint: Track which letters you've seen in a set as you loop through the (lowercased) sentence. At the end, compare the size of your set of seen letters to 26 — or check if all 26 letters are present.

Q9. Second Largest Number in a List (*Medium — Loops + Conditionals*)

Find the second largest number in a list using a single loop (no sorting).

Hint: Track two variables — `largest` and `second_largest` — and update both carefully inside the loop. Think through what happens when a new number is bigger than `largest`, versus when it's only bigger than `second_largest`.

Q10. Caesar Cipher Shift Detector (*Hard — String + Loop*)

Given an original word and its Caesar-cipher-encrypted version, determine the shift value used (assume only letters, same case).

Hint 1: Compare the first letter of both strings — the difference in their alphabet positions (using `ord()`) gives you the shift. **Hint 2:** Handle wraparound: if the difference is negative, add 26 to bring it into the valid 0–25 range. Verify the shift works for the *entire* word, not just the first letter, in case of ambiguous single-letter cases.

Q11. Run-Length Decoding (Hard — String Manipulation)

Given a run-length-encoded string like `"a3b2c1"`, expand it back to `"aaabbc"`.

Hint 1: The string alternates between a letter and a number — you need to process it two characters (or more, for multi-digit counts) at a time, not one. **Hint 2:** Use a loop with an index you control manually (a `while` loop with a manually incremented index works better here than a simple `for char in text`), since digit-runs can be more than one character long (e.g., `"a12"`).

Q12. String Rotation Checker (Hard — String Manipulation)

Check if string `B` is a rotation of string `A` (e.g., `"waterbottle"` and `"erbottlewat"` are rotations of each other) — **without using loops to manually rotate**.

Hint 1: Think about what happens when you concatenate `A` with itself (`A + A`). Does the rotated version appear somewhere inside that combined string? **Hint 2:** Use the `in` operator on `A + A` to check if `B` exists inside it — but first check that `A` and `B` are the same length, otherwise this trick gives false positives.

Q13. All Palindromic Substrings (Hard — String + Loops)

Given a string, find and print all substrings (length ≥ 2) that are palindromes.

Hint 1: You'll need **nested loops**: an outer loop for the starting index of the substring, and an inner loop for the ending index, generating every possible substring using slicing. **Hint 2:** For each generated substring, check if it equals its own reverse (`sub == sub[::-1]`) and has length ≥ 2 before printing it. Watch out for duplicate substrings if the same palindrome appears more than once.

Q14. Traffic Light Simulator using `match-case` (Medium — Switch/Conditionals)

Given a traffic light color (`"red"`, `"yellow"`, `"green"`), print the correct action (`"Stop"`, `"Get Ready"`, `"Go"`) using `match-case`. Handle invalid colors with a default case.

Hint: Structure it as `match color: case "red": ... case _: ...` — remember `case _` is your fallback for anything unexpected, similar to `else`.

Q15. Simple Calculator using `match-case` (Medium — Switch/Conditionals)

Take two numbers and an operator (`+`, `-`, `*`, `/`) as input, and use `match-case` to perform the correct operation. Handle division by zero and invalid operators.

Hint: Put the division-by-zero check *inside* the `"/"` case block itself (using a nested `if`), since `match-case` alone can't express "divide, but only if the second number isn't zero."

Q16. Find the Missing Number (Medium/Hard — Loops + Conditionals)

Given a list of numbers from `1` to `n` with exactly one missing, find the missing number using a loop (not the sum-formula shortcut) — build it with a loop and a checking condition.

Hint: Loop through the range `1` to `n`, and for each number check `if number not in the_list`. Think about what

this costs in efficiency for large lists, and consider using a `set()` for faster lookups as a refinement.

Q17. Diamond Pattern (Hard — Nested Loops)

Print a diamond shape made of `*` for a given size `n` (e.g., `n=3`):

```
  *
 ***
*****
 ***
  *
```

Hint 1: Split it into two halves — an upper pyramid (increasing rows) and a lower pyramid (decreasing rows) — each needs its own loop with a pattern for spaces before the stars and the count of stars. **Hint 2:** For row `i` (starting at 0) in the upper half, the number of leading spaces is `n-1-i` and the number of stars is `2*i+1`. Work out the mirrored version for the lower half separately.

Q18. Sentence Structure Validator (Medium — String + Conditionals)

Check whether a sentence: (a) starts with a capital letter, (b) ends with `.`, `!`, or `?`. Print which rules pass/fail.

Hint: Use `sentence[0].isupper()` for the first check, and `sentence[-1] in ".!?"` for the second — combine both results using separate `if` statements so you can report on each independently.

Q19. Number to Words Converter (0-999) (Hard — Conditionals + Strings)

Convert a number between 0 and 999 into its English words form (e.g., `452` → `"Four Hundred Fifty Two"`).

Hint 1: Break the number into hundreds, tens, and units using `//` and `%`. Handle the hundreds part first (if nonzero), then tackle the remaining two-digit part separately. **Hint 2:** The two-digit part has a tricky zone: numbers 11-19 don't follow the regular "tens word + units word" pattern (e.g., "Eleven", not "Ten One") — you'll need a special lookup or condition just for teens.

Q20. Longest Common Prefix (Hard — String + Loops)

Given a list of strings, find the longest common prefix shared by all of them (e.g., `["flower", "flow", "flight"]` → `"fl"`).

Hint 1: Start by assuming the first string in the list is the answer, then progressively shrink it by comparing against each subsequent string. **Hint 2:** For each other string, trim your current prefix guess down (character by character, or using slicing) until it actually matches the start of that string — if it shrinks to an empty string, you can stop early since there's no common prefix at all.

End of question set. Solutions with three alternative approaches per problem are provided in the companion solutions document.